

Research Notes on Softmax function

Softmax function

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

>> Section 1

The softmax function, also known as softargmax or normalized exponential function, converts a tuple of K real numbers into a probability distribution over K possible outcomes. It is a generalization of the logistic function to multiple dimensions, and is used in multinomial logistic regression. The softmax function is often used as the last activation function of a neural network to normalize the output of a network to a probability distribution over predicted output classes.

>> Section 2

In words, the softmax applies the standard exponential function to each element z_i of the input tuple $\mathbf{z}$ (consisting of K real numbers), and normalizes these values by dividing by the sum of all these exponentials. The normalization ensures that the sum of the components of the output vector $\sigma(\mathbf{z})$ is 1. The term "softmax" derives from the amplifying effects of the exponential on any maxima in the input tuple. For example, the standard softmax of $(1, 2, 8)$ is approximately $(0.001, 0.002, 0.997)$, which amounts to assigning almost all of the total unit weight in the result to the position of the tuple's maximal element (of 8). In general, instead of e a different base $b > 0$ can be used. As above, if $b > 1$ then larger input components will result in larger output probabilities, and increasing the value of b will create probability distributions that are more concentrated around the positions of the largest input values. Conversely, if $0 < b < 1$ then smaller input components will result in larger output probabilities, and decreasing the value of b will create probability distributions that are more concentrated around the positions of the smallest input values. Writing $b = e^\beta$ or $b = e^{-\beta}$ (for real β) yields the expressions:

>> Section 3

History The softmax function was used in statistical mechanics as the Boltzmann distribution in the foundational paper Boltzmann (1868), formalized and popularized in the influential textbook Gibbs (1902). The use of the softmax in decision theory is credited to R. Duncan Luce, who used the axiom of independence of irrelevant alternatives in rational choice theory to deduce

the softmax in Luce's choice axiom for relative preferences. In machine learning, the term "softmax" is credited to John S. Bridle in two 1989 conference papers, Bridle (1990a): and Bridle (1990b):

>> Section 4

$$\begin{aligned} m_{i-1} - m_i &= o_{i-1} - 1 + e^{z_i - m_i} \\ z_i - m_i &= \max(z_1, \dots, z_i) - \max(m_{i-1}, z_i) \\ &= e^{z_1 - m_{i-1}} + \dots + e^{z_i - m_{i-1}} \\ &= e^{z_i - m_i} + e^{m_{i-1} - m_i} \\ &= e^{z_i - m_i} + e^{m_{i-1} - m_i} \end{aligned}$$
 and returns $o_N / \sum_{i=1}^N o_i$. In practice, FlashAttention operates over multiple queries and keys per loop iteration, in a similar way as blocked matrix multiplication. If backpropagation is needed, then the output vectors and the intermediate arrays $[m_1, \dots, m_N]$, $[o_1, \dots, o_N]$ are cached, and during the backward pass, attention matrices are rematerialized from these, making it a form of gradient checkpointing.

>> Section 5

$$P_t(a) = \exp\left(\frac{q_t(a)}{\tau}\right) \sum_{i=1}^n \exp\left(\frac{q_t(i)}{\tau}\right)$$

>> Section 6

As $\beta \rightarrow \infty$, temperature goes to zero, $T = 1/\beta \rightarrow 0$, so eventually all differences become large (relative to a shrinking temperature), which gives another interpretation for the limit behavior.

>> Section 7

This transformation can be considered a multi-input generalisation of the logistic, operating on the whole output layer. It preserves the rank order of its input values, and is a differentiable generalisation of the 'winner-take-all' operation of picking the maximum value. For this reason we like to refer to it as softmax.

>> Section 8

$$\sigma_{1/100}(0, 10) = (1/(1 + e^{1/10}), e^{1/10}/(1 + e^{1/10})) \approx (0.475, 0.525)$$

>> Section 9

Numerical algorithms The standard softmax is numerically unstable because of large exponentiations. The safe

Research Notes on Softmax function

Softmax function

softmax method calculates instead $\sigma(\mathbf{z})_i = \frac{e^{\beta(\mathbf{z}_i - m)}}{\sum_{j=1}^K e^{\beta(\mathbf{z}_j - m)}}$ where $m = \max_i \mathbf{z}_i$ is the largest factor involved. Subtracting by it guarantees that the exponentiations result in at most 1. The attention mechanism in Transformers takes three arguments: a "query vector" $\mathbf{q}$, a list of "key vectors" $\mathbf{k}_1, \dots, \mathbf{k}_N$, and a list of "value vectors" $\mathbf{v}_1, \dots, \mathbf{v}_N$, and outputs a softmax-weighted sum over value vectors:
$$\mathbf{o} = \sum_{i=1}^N \frac{e^{\mathbf{q}^T \mathbf{k}_i - m}}{\sum_{j=1}^N e^{\mathbf{q}^T \mathbf{k}_j - m}} \mathbf{v}_i$$
 The standard softmax method involves several loops over the inputs, which would be bottlenecked by memory bandwidth. The FlashAttention method is a communication-avoiding algorithm that fuses these operations into a single loop, increasing the arithmetic intensity. It is an online algorithm that computes the following quantities: $\mathbf{z}_i = \mathbf{q}^T \mathbf{k}_i$, $m = \max(\mathbf{z}_1, \dots, \mathbf{z}_N)$

>> Section 10

We are concerned with feed-forward non-linear networks (multi-layer perceptrons, or MLPs) with multiple outputs. We wish to treat the outputs of the network as probabilities of alternatives (e.g. pattern classes), conditioned on the inputs. We look for appropriate output non-linearities and for appropriate criteria for adaptation of the parameters of the network (e.g. weights). We explain two modifications: probability scoring, which is an alternative to squared error minimisation, and a normalised exponential (softmax) multi-input generalisation of the logistic non-linearity.